Good Fences

"Good fences make good neighbors."
- Robert Frost, *Mending Wall*

Autori: Marco Canonico[1], Luca Carraturo[2], Marco Fabi[3]

Abstract
Good Fences realizza l'isolamento tra tenant in una rete spine–leaf SDN mediante un controller Ryu e regole OpenFlow. La soluzione combina VLAN, controlli anti-spoofing e proxy ARP per supportare indirizzi IPv4 sovrapposti, distribuendo il traffico su percorsi multipli tramite ECMP. L'ambiente sperimentale utilizza Mininet e Open vSwitch per verificare connettività, isolamento e distribuzione del traffico.

[1] *Canonico: DE9000168*
[2] *Carraturo: DE9000145*
[3] *Fabi: DE9000173*

Indice

1 Problema, requisiti e ipotesi 1
2 Architettura e ambiente sperimentale 1
3 Pipeline OpenFlow 2
4 Installazione & Esecuzione 5
4.1 Esecuzione tramite Docker 5
4.2 Installazione manuale 5
5 Validazione 6
5.1 Metodologia 6
5.2 Discussione dei risultati 6
6 Limiti, sviluppi futuri e conclusioni 9

1. Problema, requisiti e ipotesi

In un data center multi-tenant, più tenant, ossia clienti o organizzazioni distinti, condividono la stessa infrastruttura fisica, ma richiedono reti logicamente separate. Good Fences applica l'isolamento del traffico su una fabric spine–leaf SDN: ciascun tenant dispone di una rete logica indipendente, anche in presenza di indirizzi IPv4 sovrapposti. L'obiettivo è garantire tale separazione e distribuire il traffico tra leaf su percorsi equivalenti.

Requisiti

R1 – Isolamento: consentire la comunicazione tra host dello stesso tenant e impedire quella tra tenant diversi.

R2 – Anti-spoofing: scartare i pacchetti con MAC o IP sorgente non coerenti con il binding della porta e i frame con tag VLAN provenienti dagli host.

R3 – Indirizzi sovrapposti: consentire a tenant diversi di utilizzare gli stessi indirizzi IPv4, mantenendo la risoluzione ARP separata per tenant.

R4 – Multipath: distribuire i flussi tra leaf distinti sui percorsi equivalenti mediante ECMP.

R5 – Deny-by-default: scartare tutto il traffico non esplicitamente autorizzato, senza flooding né apprendimento dinamico.

Ipotesi e confini I binding porta–host sono statici e caricati all'avvio dalla configurazione YAML.I MAC sono univoci, assegnati dal provider, cioè dal gestore della fabric che redige la configurazione. Si assume che controller e switch della fabric siano fidati: i controlli sulle sorgenti si applicano agli ingressi dalle porte host. Ogni tenant utilizza una sola subnet e non è previsto routing tra subnet. Sono supportati esclusivamente IPv4 e ARP, con IPv6 disabilitato sugli host.

Architettura Spine-Leaf La scelta dell'architettura Spine-Leaf è motivata dalla sua capacità di gestire efficacemente il traffico East-West, cioè gli scambi tra server all'interno del datacenter. La disponibilità di più percorsi equivalenti permette di distribuire il traffico tramite ECMP, riducendo il rischio di colli di bottiglia. Il numero costante di passaggi tra Leaf diversi rende inoltre la latenza più prevedibile, mentre la struttura modulare facilita l'aggiunta di nuovi server e l'aumento della capacità di rete. Queste caratteristiche la rendono particolarmente adatta a un datacenter con molti scambi interni e necessità di crescita.

2. Architettura e ambiente sperimentale

L'ambiente sperimentale utilizza Mininet per emulare host e collegamenti, Open vSwitch per gli switch e Ryu come controller SDN. Questa combinazione consente di sperimentare

la separazione tra piano di controllo e piano dati in un ambiente riproducibile. Gli switch utilizzano OpenFlow 1.3 e sono configurati con `failMode=secure`, che impedisce il passaggio al comportamento di switching autonomo in caso di perdita del controller.

La fabric comprende due spine e tre leaf, ciascuno con quattro porte host e un collegamento verso ogni spine. I tenant A e B condividono volutamente la stessa subnet e alcuni indirizzi IPv4 per verificare l'indipendenza delle reti logiche.

La topologia spine–leaf, una rete di Clos a due livelli, è adatta al traffico di un data center, prevalentemente est–ovest, cioè tra server. Nell'architettura gerarchica a tre livelli (accesso, aggregazione e core) lo Spanning Tree disattiva i collegamenti ridondanti e il traffico tra rack risale verso i livelli superiori. Nello spine–leaf, invece, ogni leaf è collegato a tutti gli spine: due leaf qualsiasi distano sempre due salti e sono uniti da tanti percorsi equivalenti quanti sono gli spine. Tutti i collegamenti restano quindi utilizzabili tramite ECMP (R4) e la fabric cresce aggiungendo spine per la banda o leaf per le porte. In Good Fences lo Spanning Tree non serve: l'assenza di cicli è garantita dalle regole, che non ammettono transito tra uplink (T23).

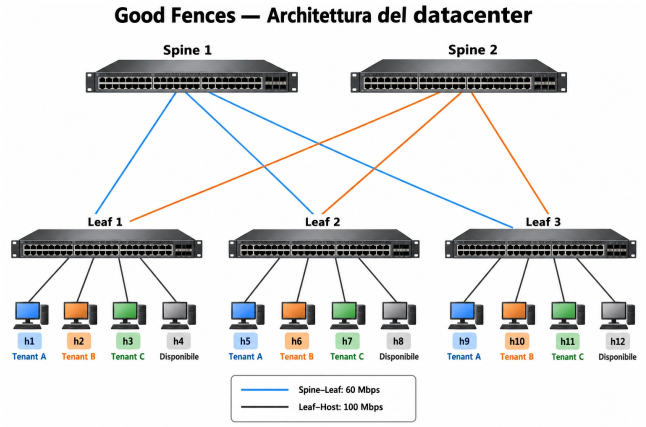


Figura 1. Topologia spine–leaf. Colori diversi per ogni tenant; spare in grigio.

Tenant	VLAN	Subnet	Host (IP)
A	101	10.0.0.0/24	h1 (.1), h5 (.2), h9 (.3)
B	102	10.0.0.0/24	h2 (.1), h6 (.2), h10 (.13)
C	103	10.0.1.0/24	h3 (.1), h7 (.2), h11 (.3)
Spare	–	–	h4, h8, h12

Tabella 1. Assegnazione dei tenant. Gli IP sono abbreviati rispetto alla subnet indicata.

3. Pipeline OpenFlow

La pipeline utilizza tre tabelle sui leaf: classificazione e controllo della sorgente (tabella 0), validazione della destinazione (tabella 1) e inoltro (tabella 2). Gli spine impiegano una sola tabella per inoltrare il traffico verso il leaf di destinazione. Ogni tabella contiene una regola di default che scarta i pacchetti non riconosciuti. Per una panoramica delle regole si rimanda alla Tabella 2. La Figura 2 riassume il percorso di un frame nelle tabelle di un leaf.

Classificazione e anti-spoofing Il binding $(\text{DPID}, \text{porta}) \rightarrow (\text{tag}, \text{host})$ identifica il tenant e le sorgenti ammesse su ciascuna porta host. I controlli sono eseguiti dalle regole OpenFlow nel piano dati: per IPv4 devono coincidere MAC e IP sorgente; per le richieste ARP devono coincidere anche SHA e SPA, ossia gli indirizzi hardware e di protocollo del mittente riportati nel messaggio ARP, che un host potrebbe falsificare lasciando corretta l'intestazione Ethernet.

La VLAN viene inserita dal leaf e non può essere scelta dall'host. Le regole a priorità maggiore scartano i frame riconosciuti come tagged e quelli con tag annidati. Il traffico privo di una regola permissiva, compreso quello untagged proveniente dalle porte spare, ricade nel table-miss.

Proxy ARP e indirizzi sovrapposti Le richieste ARP ammesse vengono inviate al controller, che ricava il tenant dal binding della porta di ingresso e cerca la destinazione mediante la chiave (tag, IP). Se l'indirizzo appartiene al tenant, il proxy genera una risposta con il MAC dell'host destinatario e la restituisce, senza tag, soltanto alla porta richiedente. Le destinazioni sconosciute non ricevono risposta e le richieste non vengono diffuse nella fabric.

La risoluzione rimane quindi indipendente anche con indirizzi sovrapposti: una richiesta per `10.0.0.2` proveniente da h1 risolve il MAC di h5, mentre la stessa richiesta da h2 risolve quello di h6.

Forwarding IPv4 ed ECMP Dopo la classificazione, la tabella 1 verifica che MAC e IP di destinazione appartengano al tenant indicato dalla VLAN. Il tag conserva quindi l'identità del tenant sorgente lungo la fabric, anche se i MAC sono globalmente univoci. La tabella 2 rimuove il tag e consegna il pacchetto se la destinazione è locale (interna alla leaf); altrimenti seleziona un uplink tramite il gruppo ECMP.

Ogni leaf dispone del gruppo SELECT 1, composto da due bucket di peso 1, associati agli uplink 5 e 6, ciascuno collegato a uno spine diverso. La regola con cookie 51 rimanda al gruppo e la scelta del bucket è compiuta da Open vSwitch nel piano dati, non dal controller; per ciascun pacchetto viene scelto un solo bucket, senza duplicazione.

La selezione avviene per flusso e non per pacchetto. Il controller non specifica un metodo di selezione, quindi Open vSwitch (dalla versione 2.10) applica quello predefinito, `dp_hash`: calcola un hash sulla quintupla del pacchetto (indirizzi IP, protocollo e porte L4) e associa ogni valore a un bucket in proporzione ai pesi. I pacchetti di uno stesso flusso producono lo stesso hash e seguono lo stesso percorso: si evita così il riordino che una scelta per pacchetto provocherebbe, ad esempio in TCP.

Il bilanciamento riguarda quindi i flussi e non i pacchetti: con pesi uguali ogni flusso ha la stessa probabilità di essere

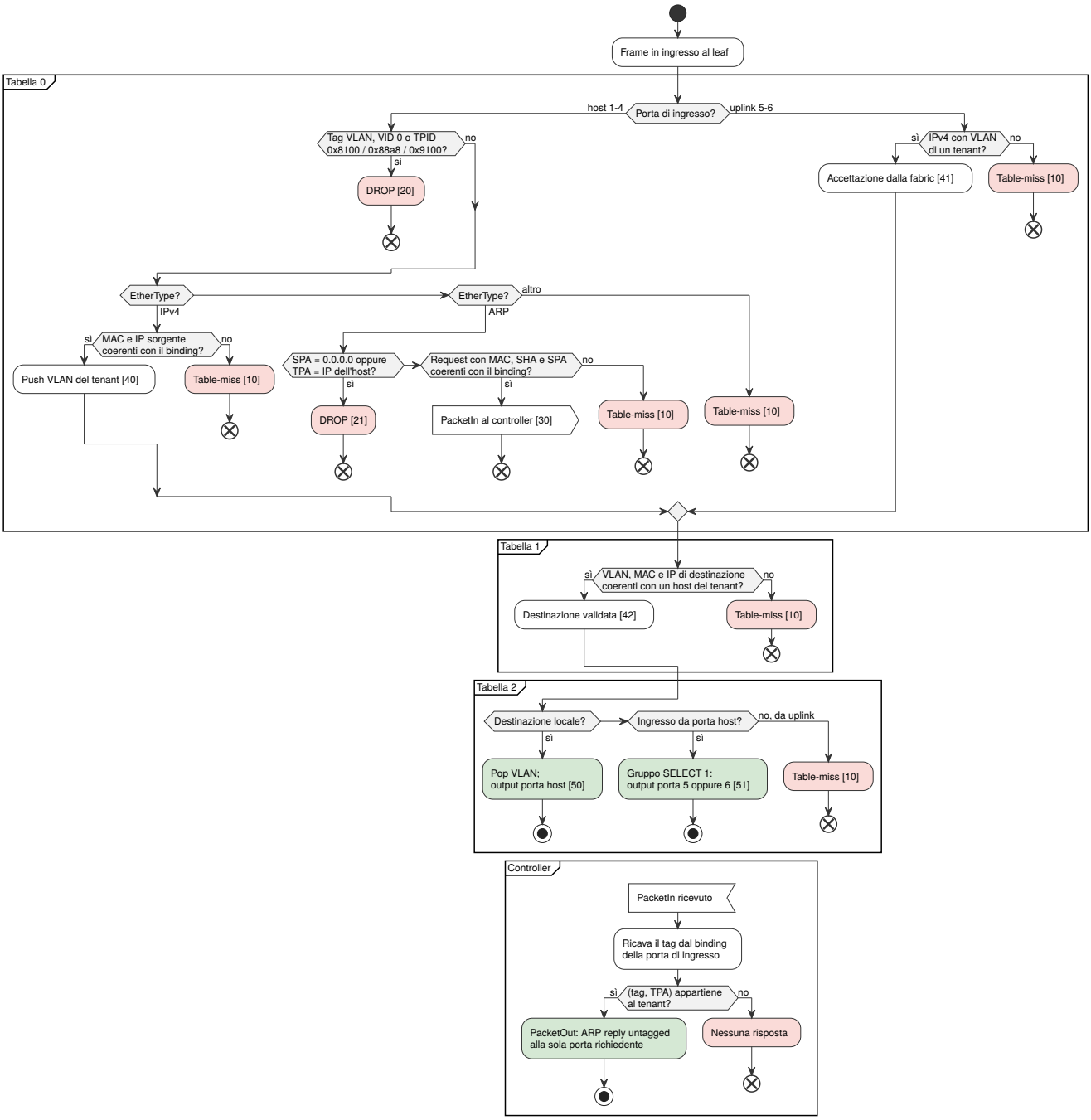


Figura 2. Elaborazione di un frame nella pipeline di un leaf. Le decisioni seguono l'ordine di priorità delle regole: si tratta di un equivalente logico, poiché OpenFlow applica direttamente la regola corrispondente di priorità più alta. Tra parentesi quadre i suffissi dei cookie (Tabella 2). Ogni ramo non autorizzato termina in uno scarto; la regola di scarto ARP della tabella 1 [21], non raggiungibile, è omessa.

Sw/T	Prio	Match	Istruzioni	xx
Leaf/0	400	Porta host, VLAN presente (anche VID 0) oppure EtherType `0x8100`, `0x88a8`, `0x9100`	DROP	20
Leaf/0	350	Porta assegnata, ARP untagged, SPA nullo oppure TPA uguale all'IP dell'host associato alla porta	DROP	21
Leaf/0	300	ARP request untagged; porta, MAC Ethernet, SHA e SPA coerenti con il binding	Invio del frame completo al controller	30
Leaf/0	200	IPv4 untagged; porta, MAC e IP sorgente coerenti con il binding	Inserimento VLAN del tenant; goto 1	40
Leaf/0	200	Uplink 5 o 6, IPv4 con VLAN nota	Goto 1	41
Leaf/1	300	ARP	DROP	21
Leaf/1	200	IPv4, VLAN, MAC e IP di destinazione coerenti con un host del tenant	Goto 2	42
Leaf/2	200	VLAN del tenant, MAC di destinazione locale	Rimozione VLAN; output porta host	50
Leaf/2	200	Porta host assegnata al tenant, VLAN del tenant, MAC di destinazione remota	Gruppo SELECT 1: output 5 oppure 6	51
Spine/0	200	Ingresso da un leaf diverso da quello di destinazione; IPv4, VLAN, MAC e IP di destinazione coerenti	Output verso il leaf di destinazione	52
Tutte	0	Table-miss	DROP	10

Tabella 2. Regole della pipeline. L'ultima colonna riporta il suffisso esadecimale del cookie `0x474600xx`, che identifica la categoria della regola (si veda il paragrafo *Identificazione delle regole*).

assegnato a ciascun uplink, ma su un numero finito di flussi la ripartizione si discosta da 50/50, e un singolo flusso resta sempre sullo stesso uplink.

Il percorso remoto è host → leaf sorgente → spine → leaf destinazione → host. Gli spine preservano la VLAN e inoltrano verso il leaf corretto, escludendo il ritorno sulla porta di ingresso.

L'uso delle VLAN mantiene semplice l'identificazione dei tenant nella fabric emulata; un overlay VXLAN costituisce un possibile sviluppo per estendere la soluzione oltre questo perimetro (Sezione 6).

Identificazione delle regole Ogni regola installata dal controller riporta un cookie della forma `0x474600xx`.
Nell'ultimo byte, la cifra esadecimale più significativa indica la famiglia della regola: 1 per il table-miss, 2 per gli scarti di sicurezza, 3 per l'invio al controller, 4 per la classificazione e la validazione, 5 per l'inoltro. La cifra meno significativa distingue le varianti all'interno della stessa famiglia e non corrisponde al numero di tabella: il cookie 21, ad esempio, è condiviso dalle regole di scarto ARP delle tabelle 0 e 1. Il cookie non partecipa al matching, ma associa ciascun contatore OpenFlow a una categoria di regole, come avviene nella validazione (Sezione 5), e consente di selezionare un'intera famiglia mediante maschera, ad esempio `cookie=0x47460040/0xfffffff0` con `ovs-ofctl`.

Installazione e stato READY Alla connessione, il controller verifica il DPID atteso, l'utilizzo di OpenFlow 1.3 e la disponibilità delle tabelle necessarie. L'installazione procede quindi come segue:

1. Rimozione dei flow precedenti e installazione dei table-miss DROP. Sui leaf viene eliminato e ricreato il gruppo ECMP.

2. Sui leaf, attesa di una prima Barrier Reply prima di inviare i flow che utilizzano il gruppo. Gli spine, privi di gruppi ECMP, installano direttamente i flow dopo la preparazione.

3. Invio di una Barrier finale e passaggio dello switch allo stato `READY` dopo la relativa risposta. Errori OpenFlow o un timeout di 10 secondi per la fase in attesa causano la chiusura della connessione.

La Figura 3 riporta lo scambio di messaggi per un leaf.

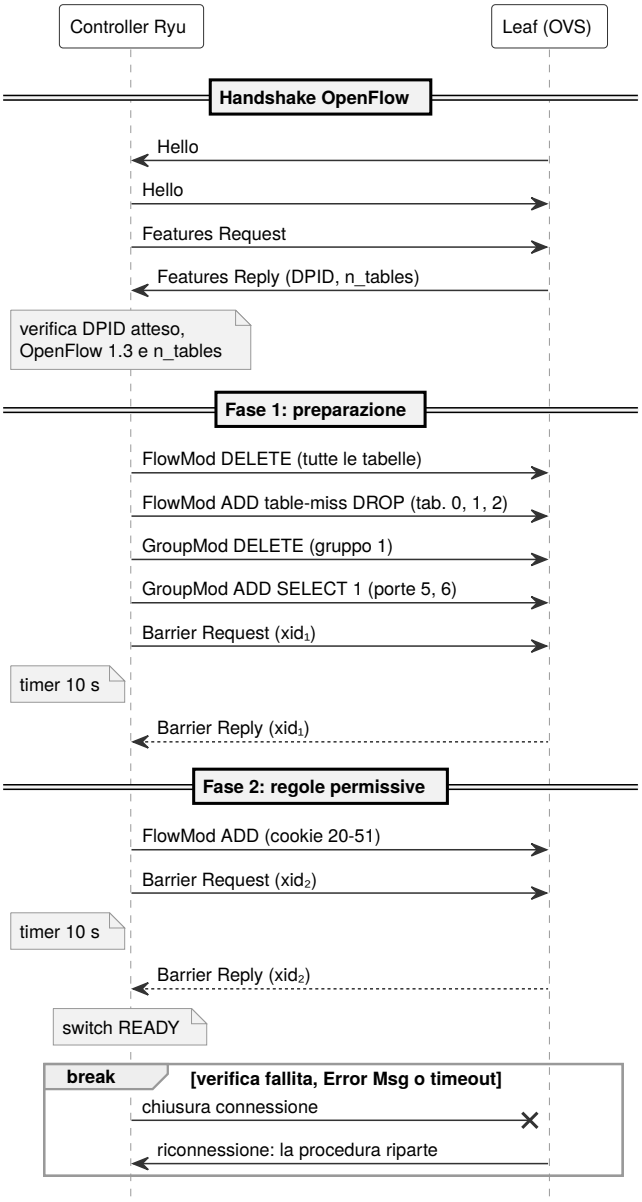


Figura 3. Installazione della pipeline su un leaf. La prima Barrier garantisce che il gruppo ECMP esista prima dell'invio dei flow che lo richiamano. Sugli spine il GroupMod è assente e le regole permissive seguono direttamente la preparazione, con la sola Barrier finale.

Alla riconnessione la procedura viene ripetuta, ricostruendo le regole dalla configurazione caricata dal controller.

4. Installazione & Esecuzione

Il progetto supporta due modalità di esecuzione: tramite container Docker, soluzione consigliata per semplificare la gestione delle dipendenze, oppure mediante installazione manuale nell'ambiente host.

4.1 Esecuzione tramite Docker

L'esecuzione tramite Docker richiede Docker Engine e Docker Compose v2. Il Dockerfile definisce un'immagine basata su Python 3.9 che include Mininet, Open vSwitch, il controller Ryu e le dipendenze della dashboard.

Dalla directory principale del repository, il container può essere costruito e avviato mediante:

```
docker compose up -d --build
```

Il file `docker-compose.yml` configura un container privilegiato, necessario per la gestione dei namespace di rete e degli switch virtuali, ed espone la dashboard sull'indirizzo `http://127.0.0.1:8501`.

Una volta aperta l'interfaccia web, il pulsante *Avvia rete* permette di avviare il controller Ryu e la topologia Mininet.

In alternativa alla dashboard, è possibile avviare separatamente controller e rete all'interno del container. I seguenti comandi vanno eseguiti in due terminali distinti:

```
docker compose exec good-fences \
  bash scripts/start-controller.sh
```

```
docker compose exec good-fences \
  bash scripts/start-network.sh
```

Nel secondo terminale viene resa disponibile la CLI di Mininet, attraverso cui eseguire le verifiche di connettività. La funzione grafica *Apri shell*, basata su xterm, non è prevista nella modalità Docker ma sarà oggetto di implementazioni future.

Per terminare la simulazione avviata dalla dashboard, utilizzare *Arresta rete* e successivamente:

```
docker compose down
```

4.2 Installazione manuale

In alternativa, Good-Fences può essere eseguito direttamente su di un host Linux. L'esecuzione della rete necessita di privilegi amministrativi.

Dalla radice del repository, creare un ambiente virtuale Python e installare le dipendenze:

```
python3.9 -m venv .venv
.venv/bin/python -m pip install --upgrade pip
.venv/bin/python -m pip install \
  --build-constraint build-constraints.txt \
  -r requirements.txt
```

Il controller Ryu e la rete Mininet possono essere avviati in due terminali separati:

```
bash scripts/start-controller.sh
```

```
bash scripts/start-network.sh
```

In alternativa, la dashboard può essere avviata tramite:

```
bash scripts/start-visualizer.sh
```

La dashboard è accessibile all'indirizzo `http://127.0.0.1:8501` e consente di gestire l'avvio e l'arresto della simulazione, eseguire i casi di test e consultare i contatori OpenFlow.

Prima di avviare la simulazione dalla dashboard, è necessario terminare eventuali sessioni Mininet già attive e arrestare il controller avviato manualmente, evitando conflitti tra processi.

5. Validazione

Il processo di validazione verifica i requisiti R1–R5 sulla configurazione della Tabella dei tenant. I casi di test inviano un frame costruito con l'utility `tests/network/packet_io.py`, basata su Scapy (libreria Python per la costruzione di pacchetti), dalla CLI di Mininet oppure dalla pagina *Esecuzione test* della dashboard (Figura 4); fanno eccezione i casi *trace* (T5, T19, T23, T24), simulati con `ofproto/trace` e l'ispezione delle tabelle (T25).

5.1 Metodologia

Le statistiche dei flow mostrate dalla dashboard sono contatori cumulativi dall'avvio della rete. Le Figure 5–7 provengono da un'unica sessione in cui T1, T6–T10 e T11 sono stati eseguiti in quest'ordine: l'effetto di ciascun test è la differenza rispetto alla figura precedente, mentre per T1, primo test della sessione, le regole del percorso partono da zero. Per i gruppi ECMP, invece, la funzione *Salva riferimento prima del test* della pagina *Statistiche* mostra direttamente il delta dei contatori dei bucket (Figura 8).

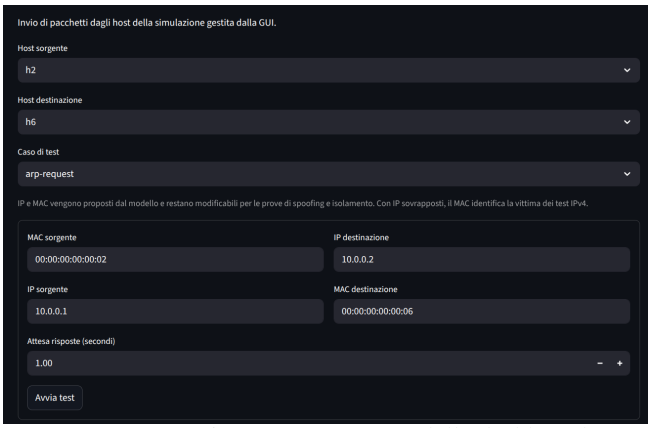


Figura 4. Pagina *Esecuzione test* della dashboard

5.2 Discussione dei risultati

La Tabella 3 raccoglie i casi di test, raggruppati per requisito. Di seguito si descrive cosa verifica ciascun gruppo e quali evidenze lo dimostrano.

R1 – Isolamento T1 è il caso positivo di riferimento: un frame IPv4 valido da h2 a h6, entrambi del tenant B e collegati a leaf diversi, attraversa l'intera pipeline.
La Tabella 4 riporta le regole che devono incrementare lungo il percorso: il leaf sorgente inserisce la VLAN 102 e inoltra tramite il gruppo ECMP, lo spine selezionato preserva il tag e il leaf di destinazione lo rimuove prima della consegna. T2 e T3 tentano la comunicazione tra i tenant A e B con una sorgente legittima: il frame supera la tabella 0, ma la tabella 1 non trova la destinazione nel tenant indicato dalla VLAN e il pacchetto termina nel table-miss. Poiché h5 e h6 condividono l'indirizzo 10.0.0.2, il frame usa il MAC della vittima: un ping da h1 verso h6 risolverebbe infatti h5 e non costituirebbe un attacco. T4 verifica che la destinazione sia validata come coppia MAC/IP, anche quando entrambi gli indirizzi appartengono al tenant. T5 mostra che lo spine ripete la verifica: un frame con VLAN 101 diretto a un host di B non ha regole di inoltro.

Switch	Tab.	Cookie	Δ att.	Δ oss.
leaf1	0	40	1	1
leaf1	1	42	1	1
leaf1	2	51	1	1
spine1/2	0	52	1	1
leaf2	0	41	1	1
leaf2	1	42	1	1
leaf2	2	50	1	1

Tabella 4. Incremento dei contatori lungo il percorso di T1. Il delta dello spine compare su uno solo dei due. La risposta ICMP *Protocol Unreachable* di h6 incrementa anche le regole del percorso inverso, visibili su leaf1 nella Figura 5 (41, 50 e la regola 42 verso h2).



Figura 5. Vista *Rete* della pagina *Statistiche*: contatori cumulativi delle regole dopo T1, primo test della sessione.

R2 – Anti-spoofing T6 e T7 alterano rispettivamente il MAC e l'IP sorgente di un frame altrimenti valido: nessuna regola della tabella 0 corrisponde al binding della porta e il pacchetto ricade nel table-miss, senza raggiungere la tabella 1. T8 e T9 tentano di scegliere la VLAN: tag singolo, VID 0,

ID	Req.	Sorg. → dest.	Frame inviato	Risultato atteso	Risultato osservato	Esito
T1	R1	h2 → h6	IPv4 untagged, sorgente e destinazione coerenti (tenant B, leaf diversi)	h6 riceve 1 frame untagged; leaf1: 40, 42, 51; spine: 52; leaf2: 41, 42, 50	1 frame ricevuto da h6; cookie e percorso conformi	**PASS**
T2	R1	h1 → h6	IPv4 da h1 (A) con MAC e IP di h6 (B)	Miss tab. 1 su leaf1; nessun frame su h5 e h6	Miss tab. 1 rilevato; nessun frame ricevuto	**PASS**
T3	R1, R3	h2 → h5	IPv4 da h2 (B) con MAC di h5 (A) e IP sovrapposto 10.0.0.2	Miss tab. 1 su leaf1; nessun frame su h5	Miss tab. 1 rilevato; nessun frame ricevuto	**PASS**
T4	R1	h2 → h6	IPv4 con MAC di h6 e IP di h10 (10.0.0.13)	Miss tab. 1: coppia MAC/IP incoerente; nessun frame su h6	Coppia MAC/IP rifiutata; nessun frame ricevuto	**PASS**
T5	R1	spine1 (trace)	IPv4 con VLAN 101 e MAC/IP di h6, ingresso da leaf1	Table-miss: nessun output verso leaf2	Table-miss confermato; nessun output	**PASS**
T6	R2	h2 → h6	IPv4 con MAC sorgente di h1	Miss tab. 0; cookie 40 invariato; nessun frame su h6	Miss tab. 0; cookie 40 invariato; nessuna consegna	**PASS**
T7	R2	h2 → h6	IPv4 con IP sorgente 10.0.0.99	Miss tab. 0; nessun frame su h6	Miss tab. 0 rilevato; nessuna consegna	**PASS**
T8	R2	h2 → h6	IPv4 con tag VLAN 102, 101 oppure VID 0	DROP priorità 400 (cookie 20)	Tutti i frame taggati scartati; cookie 20	**PASS**
T9	R2	h2 → h6	IPv4 con doppio tag 101/102 oppure TPID `0x88a8`, `0x9100`	DROP priorità 400 (cookie 20)	Doppi tag e TPID alternativi scartati	**PASS**
T10	R2	h2 → bcast	ARP request con MAC Ethernet, SHA oppure SPA falsificati	Miss tab. 0; nessun PacketIn né reply	Miss tab. 0; nessun PacketIn o reply	**PASS**
T11	R3	h2 → bcast	ARP request per 10.0.0.2	Reply untagged, solo verso h2, con MAC di h6; cookie 30; nessun ARP sugli uplink	Reply corretta a h2; cookie 30; uplink senza ARP	**PASS**
T12	R3	h1 → bcast	ARP request per 10.0.0.2	Reply untagged, solo verso h1, con MAC di h5	Reply a h1 con MAC di h5; nessuna diffusione	**PASS**
T13	R3	h1 → bcast	ARP request per 10.0.0.13 (presente solo in B)	Nessuna reply; cache di h1 `INCOMPLETE`, poi `FAILED`	Nessuna reply; cache `INCOMPLETE/FAILED`	**PASS**
T14	R3	h2 → bcast	ARP probe (SPA 0.0.0.0)	DROP priorità 350 (cookie 21); nessuna reply	ARP probe scartato; cookie 21; nessuna reply	**PASS**
T15	R3	h2 → bcast	Gratuitous ARP (SPA = TPA = 10.0.0.1)	DROP priorità 350 (cookie 21); nessuna reply	Gratuitous ARP scartato; cookie 21	**PASS**
T16	R4	h2 → h6	100 flussi UDP (porte sorgente 40000–40099, destinazione 9000), 10 pacchetti ciascuno	Entrambi i bucket del gruppo 1 di leaf1 con $\Delta\text{pkt} > 0$	Due bucket attivi su leaf1; $\Delta\text{pkt} > 0$	**PASS**
T17	R4	h6 → h10	Come T16	Come T16, sul gruppo di leaf2	Due bucket attivi su leaf2; $\Delta\text{pkt} > 0$	**PASS**
T18	R4	h10 → h2	Come T16	Come T16, sul gruppo di leaf3	Due bucket attivi su leaf3; $\Delta\text{pkt} > 0$	**PASS**
T19	R4	spine1/2, leaf2 (trace)	IPv4 con VLAN 102 verso h6, attraverso ciascuno spine	Esito identico sui due percorsi: output verso leaf2, rimozione VLAN e output porta 2	Entrambi i percorsi validi; VLAN rimossa su leaf2	**PASS**
T20	R5	h4 → h6	IPv4 untagged dallo spare h4	Miss tab. 0; nessun frame su h6	Traffico spare scartato; nessuna consegna	**PASS**
T21	R5	h2 → bcast	RARP request (EtherType `0x8035`)	Miss tab. 0; nessuna reply	Miss tab. 0; nessuna risposta RARP	**PASS**
T22	R5	h2 → bcast	ARP reply spontanea (opcode 2)	Miss tab. 0; nessun PacketIn	Miss tab. 0; nessun PacketIn generato	**PASS**
T23	R5	leaf1 (trace)	IPv4 con VLAN 102 verso h6 in ingresso dall'uplink 5	Miss tab. 2: nessun transito uplink → uplink	Miss tab. 2; nessun inoltro tra uplink	**PASS**
T24	R5	leaf1 (trace)	ARP con VLAN 102 in ingresso dagli uplink 5 e 6	Miss tab. 0; nessun output verso il controller	Miss tab. 0; nessun inoltro al controller	**PASS**
T25	R5	tutti gli switch	Ispezione di flow e gruppi installati	Nessuna azione NORMAL o FLOOD; un solo gruppo SELECT per leaf	NORMAL/FLOOD assenti; un gruppo SELECT per leaf	**PASS**

Tabella 3. Matrice dei test di validazione. Nella colonna del risultato atteso i numeri sono i suffissi dei cookie della Tabella 2; *miss tab. n* indica il table-miss DROP della tabella n. I casi indicati con *trace* sono simulati con `ofproto/trace`. Tutti i 25 test hanno prodotto risultati conformi alle aspettative, confermando il soddisfacimento dei requisiti R1–R5 nella configurazione esaminata.

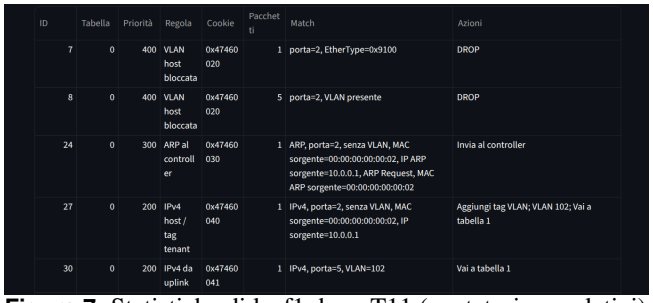
doppio tag e TPID alternativi vengono intercettati dalle regole a priorità 400 prima di ogni altro controllo. T10 applica gli stessi controlli ad ARP: la regola verso il controller richiede la coerenza di MAC Ethernet, SHA e SPA, perciò una richiesta falsificata non genera PacketIn né risposta. In tutti i casi la regola IPv4 legittima di h2 (cookie 40) non deve incrementare (Figura 6).



ID	Tabella	Priorità	Regola	Cookie	Pacchetti	Match	Azioni
7	0	400	VLAN host bloccata	0x4746020	1	porta=2, EtherType=0x9100	DROP
8	0	400	VLAN host bloccata	0x4746020	5	porta=2, VLAN presente	DROP
27	0	200	IPv4 host / tag tenant	0x4746040	1	IPv4, porta=2, senza VLAN, MAC sorgente=00:00:00:00:00:02, IP sorgente=10.0.0.1	Aggiungi tag VLAN; VLAN 102; Vai a tabella 1
30	0	200	IPv4 da uplink	0x4746041	1	IPv4, porta=5, VLAN=102	Vai a tabella 1
35	0	0	Default DROP	0x4746010	83	qualsiasi pacchetto (table-miss)	DROP
40	1	200	IPv4 destinazione validata	0x4746042	1	IPv4, VLAN=102, MAC destinazione=00:00:00:00:00:02, IP destinazione=10.0.0.1	Vai a tabella 2
41	1	200	IPv4 destinaz	0x4746042	1	IPv4, VLAN=102, MAC destinazione=00:00:00:00:00:06, IP	Vai a tabella 2

Figura 6. Statistiche di leaf1 dopo T6–T10 (contatori cumulativi). Rispetto alla Figura 5 incrementano soltanto il table-miss e le regole DROP dei frame tagged (cookie 20); i valori delle regole 40–42 derivano da T1.

R3 – Indirizzi sovrapposti T11 e T12 sono il nucleo della prova: la stessa richiesta ARP per 10.0.0.2 riceve risposte diverse in base alla porta di ingresso, il MAC di h6 per h2 e quello di h5 per h1, perché il proxy cerca la destinazione con la chiave (tag, IP) ricavata dal binding. La risposta è untagged e inviata solo alla porta richiedente: le catture sugli uplink di leaf1 e su un'altra porta host non devono mostrare ARP inoltrati. T13 richiede un indirizzo presente soltanto nel tenant B: il proxy non risponde e la voce nella cache di h1 resta incompleta. T14 e T15 coprono le ARP che non chiedono una risoluzione: probe e gratuitous ARP sono scartati in tabella 0 e non raggiungono il controller. Il contatore della regola verso il controller (cookie 30) conta le richieste ricevute, non le risposte: queste si verificano dall'output dell'utility, mentre la figura Figura 7 mostra il passaggio della richiesta sulla regola verso il controller. Sul piano dati, T3 completa la prova mostrando che l'IP sovrapposto non permette di raggiungere l'host dell'altro tenant.



ID	Tabella	Priorità	Regola	Cookie	Pacchetti	Match	Azioni
7	0	400	VLAN host bloccata	0x4746020	1	porta=2, EtherType=0x9100	DROP
8	0	400	VLAN host bloccata	0x4746020	5	porta=2, VLAN presente	DROP
24	0	300	ARP al controller	0x4746030	1	ARP, porta=2, senza VLAN, MAC sorgente=00:00:00:00:00:02, IP ARP sorgente=10.0.0.1, ARP Request, MAC ARP sorgente=00:00:00:00:00:02	Invia al controller
27	0	200	IPv4 host / tag tenant	0x4746040	1	IPv4, porta=2, senza VLAN, MAC sorgente=00:00:00:00:00:02, IP sorgente=10.0.0.1	Aggiungi tag VLAN; VLAN 102; Vai a tabella 1
30	0	200	IPv4 da uplink	0x4746041	1	IPv4, porta=5, VLAN=102	Vai a tabella 1

Figura 7. Statistiche di leaf1 dopo T11 (contatori cumulativi). Rispetto alla Figura 6 incrementa la regola verso il controller (cookie 30): un passaggio, la richiesta ARP di h2.

R4 – Multipath Un singolo flusso può restare sempre sullo stesso uplink, quindi un ping non è una prova del bilanciamento. T16–T18 generano da ciascun leaf 100 flussi UDP che differiscono per porta sorgente, con 10 pacchetti ciascuno, verso un host remoto dello stesso tenant. Il criterio è un delta pacchetti positivo su entrambi i bucket del gruppo SELECT 1; la quota di ciascun bucket è il rapporto tra il suo delta e la somma dei delta del gruppo (Tabella 5, Figura 8). Una ripartizione diversa da 50/50 è compatibile con il requisito, perché la selezione avviene per flusso (Sezione 3). T19 completa la prova: entrambi gli spine ed entrambi gli ingressi uplink del leaf di destinazione producono la stessa consegna, quindi ciascun bucket corrisponde a un percorso valido.

Leaf (flusso)	Inviati	Bucket (porta)	Δpkt	Quota
leaf1 (h2 → h6)	1000	0 (5)	450	45,0 %
		1 (6)	550	55,0 %
leaf2 (h6 → h10)	1000	0 (5)	520	52,0 %
		1 (6)	480	48,0 %
leaf3 (h10 → h2)	1000	0 (5)	420	42,0 %
		1 (6)	580	58,0 %

Tabella 5. Distribuzione sui bucket ECMP in T16–T18. Traffico offerto: 100 flussi × 10 pacchetti per leaf. Versione di Open vSwitch: 3.3.9.

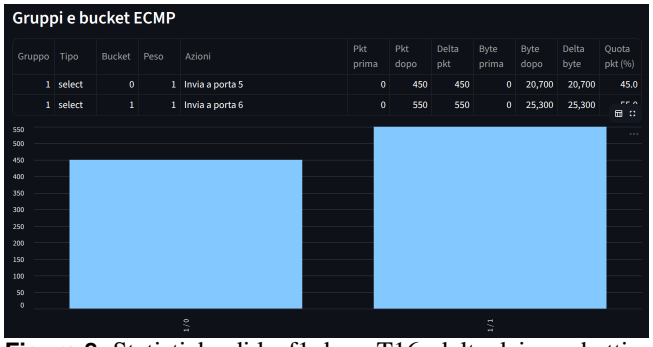


Figura 8. Statistiche di leaf1 dopo T16: delta dei pacchetti sui due bucket del gruppo SELECT 1.

R5 – Deny-by-default I casi precedenti mostrano già che il traffico non autorizzato termina in un DROP; T20–T25

verificano gli ingressi per cui nessuna regola permissiva è prevista. Lo spare h4 invia un frame ben formato ma privo di binding, che nessuna regola della tabella 0 ammette (T20). RARP e ARP reply spontanee non hanno regole dedicate e non raggiungono il controller (T21, T22). Con il trace sugli uplink, un IPv4 diretto a un host non locale termina nel table-miss della tabella 2, perché le regole ECMP ammettono soltanto ingressi dalle porte host: non esiste transito uplink → uplink e non possono formarsi cicli (T23). Un ARP proveniente dalla fabric viene scartato in tabella 0 (T24). Infine, l'ispezione delle tabelle (T25) conferma l'assenza di azioni NORMAL e FLOOD: ogni inoltro deriva da regole statiche calcolate dalla configurazione.

6. Limiti, sviluppi futuri e conclusioni

La configurazione è statica: le modifiche a tenant e binding richiedono il riavvio dell'ambiente per essere applicate. In prospettiva, un'autenticazione degli host sul modello di IEEE 802.1X permetterebbe di associare il tenant alla porta solo dopo l'autenticazione, superando i binding statici. Non sono implementate politiche di QoS: il campo `rate_mbps`, presente nel file YAML e nel modello, costituisce una predisposizione e non impone limiti di banda. Le richieste ARP ammesse generano PacketIn senza rate limiting; un host che utilizzi sorgenti valide può quindi produrre un carico elevato sul controller. Inoltre, il proxy ARP risponde in base alla sola configurazione e non verifica che il destinatario sia attivo: se l'host è spento, la risoluzione riesce comunque e i pacchetti successivi vengono persi. La risposta potrebbe essere subordinata allo stato della porta, che il controller riceve tramite i messaggi OpenFlow Port Status. I gruppi ECMP non verificano la disponibilità degli uplink: in caso di guasto, il traffico assegnato al bucket interessato può essere perso. Nella modalità Docker, la funzione *Apri shell* della dashboard non è al momento disponibile (Sezione 4). Infine, l'associazione di una VLAN distinta a ciascun tenant limita lo spazio degli identificativi a 4094 valori, senza implicare che tale scala sia stata verificata sperimentalmente; l'overlay VXLAN citato nella Sezione 3, con identificativi a 24 bit, supererebbe questo limite.

Ci si propone quindi di implementare queste modifiche in una versione futura del progetto.